Conflict serializability

Transaction Schedules

T1	T2
begin	
read(A)	
write(A)	
read(B)	
write(B)	
commit	
	begin
	read(A)
	write(A)
	read(B)
	write(B)
	commit

A **schedule** is a sequence of actions on data from one or more transactions.

Actions: Begin, Read, Write, Commit and Abort.

$R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$, $R_2(A)$, $W_2(A)$, $R_2(B)$, $W_2(B)$

By convention we only include committed transactions, and omit Begin and Commit.

Serial Equivalence

- **Definition: Serial schedule**
  - Each transaction runs from start to finish without any intervening actions from other transactions
- **Definition:** 2 schedules are **equivalent** if they:
  - involve same transactions
  - each individual transaction's actions are ordered the same
  - both schedules **leave the DB in the same final state**

For example, suppose A starts at `10`, T1 **adds 5**, and T2 **doubles it**. The left and right schedules leave A at `(10 + 5) × 2 = 30`. Assume the same for B. Hence, these two schedules are equivalent

$R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$, $R_2(A)$, $W_2(A)$, $R_2(B)$, $W_2(B)$ = $R_1(A)$, $W_1(A)$, $R_2(A)$, $W_2(A)$, $R_1(B)$, $W_1(B)$, $R_2(B)$, $W_2(B)$

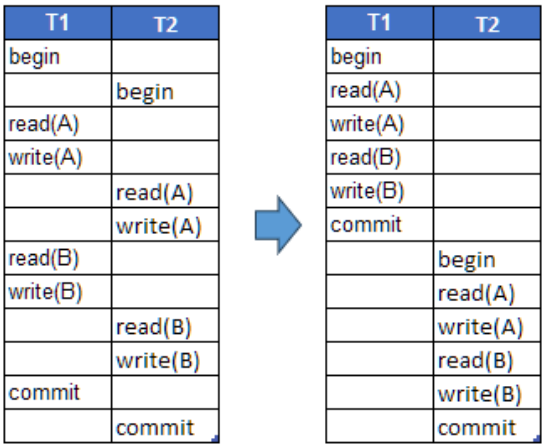
T1	T2
begin	
read(A)	
write(A)	
read(B)	
write(B)	
commit	
	begin
	read(A)
	write(A)
	read(B)
	write(B)
	commit

T1	T2
begin	
read(A)	
write(A)	
	begin
	read(A)
	write(A)
read(B)	
write(B)	
commit	
	read(B)
	write(B)
	commit

A Schedule S is **serializable** if:

- S is equivalent to some serial schedule

In a **serial schedule**, one transaction runs from beginning to end before the next one begins.



T1	T2
begin	
	begin
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)
commit	
	commit

T1	T2
begin	
read(A)	
write(A)	
read(B)	
write(B)	
commit	
	begin
	read(A)
	write(A)
	read(B)
	write(B)
	commit

Conflicting Operations

- Tricky to check property **"leaves the DB in the same final state"**
- Need an easier equivalence test!
  - Settle for a "conservative" test: always true positives, but some false negatives
  - i.e. sacrifice some concurrency for easier correctness check
- **Use notion of "conflicting" operations (read/write)**
- **Definition: Two operations conflict if they:**
  - Are by different transactions,
  - Are on the same object,
  - At least one of them is a write.
- The order of non-conflicting operations has no effect on the final state of the database!

Conflict Equivalent

Two schedules are **conflict equivalent** if they contain the same actions from the same transactions and preserve the relative order of every **conflicting pair**. Two actions conflict when they belong to different transactions, access the same data item, and at least one is a write.

Consider these two schedules, where `R` means read and `W` means write.

Schedule	Actions from left to right
S_1	$W_1(A), W_1(B), R_2(A)$
S_2	$W_1(A), R_2(A), W_1(B)$

S_1 and S_2 are conflict equivalent. The only conflicting pair is **$W_1(A)$ and $R_2(A)$**. In both schedules, T1 writes A before T2 reads it. Moving T2's read of A ahead of T1's write to B is fine because those actions access different items.

Now consider:

Schedule	Actions from left to right
S_1	$W_1(A), W_1(B), R_2(A)$
S3	$R_2(A), W_1(A), W_1(B)$

S_3 reverses the conflicting pair, so it is **not conflict equivalent** to S_1. In S_3, T2 reads A before T1 updates it.

Conflict Serializable

- **Definition: Schedule S is *conflict serializable* if:**
  - S is conflict equivalent to some serial schedule
  - Implies S is also Serializable

Note: some serializable schedules are NOT conflict serializable

- Conflict serializability gives false negatives as a test for serializability!
- The cost of a conservative test
- A price we pay to achieve efficient enforcement

Example:

Here's an example expanded to **four transactions**. T1 writes A and reads B, and T2 reads A and writes C, T3 reads C and writes D, and T4 reads D.

Schedule	Operations from left to right
S1 — serial	`W1(A), R1(B), R2(A), W2(C), R3(C), W3(D), R4(D)`
S2 — interleaved	`W1(A), R2(A), W2(C), R3(C), R1(B), W3(D), R4(D)`

Both schedules contain the same operations and preserve each transaction's internal order. They also preserve the order of **every conflicting pair**.

Data item	Order in both schedules	Precedence edge
A	`W1(A)` before `R2(A)`	T1 → T2
C	`W2(C)` before `R3(C)`	T2 → T3
D	`W3(D)` before `R4(D)`	T3 → T4

In S2, `R1(B)` moves past three operations on A and C. Since T1 is the only transaction that accesses B, those swaps preserve all conflict orders. So **S1 and S2 are conflict equivalent**, and S2 is conflict serializable in the order **T1 → T2 → T3 → T4**.

S1 (Serial)

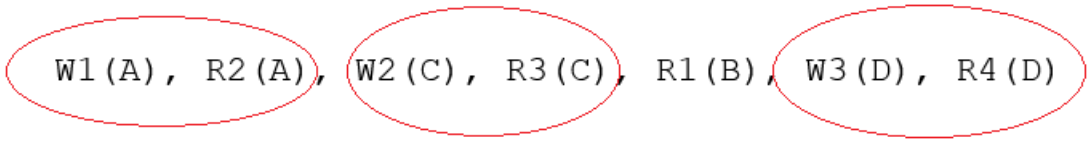
T1	T2	T3	T4
W(A)			
R(B)			
	R(A)		
	W(C)		
		R(C)	
		W(D)	
			R(D)



S2 (Interleaved)

T1	T2	T3	T4
W(A)			
	R(A)		
	W(C)		
		R(C)	
R(B)			
		W(D)	
			R(D)

The predence graph is acyclic. T1 → T2 → T3 → T4



A schedule S is conflict serializable if

- You are able to transform S into a serial schedule by swapping consecutive non-conflicting operations of different transactions

T1	R(A)	W(A)			R(B)	W(B)		
T2			R(A)	W(A)			R(B)	W(B)

T1	R(A)	W(A)	R(B)	W(B)				
T2					R(A)	W(A)	R(B)	W(B)

- Here's another example:

T1	R(A)			W(A)
T2		R(A)	W(A)	

Is this conflict serializable or not?

R1(A), R2(A), W2(A), W1(A)

A **schedule** is the order in which a database executes operations from one or more transactions. A schedule is **conflict serializable** if you can rearrange it into a serial schedule by swapping adjacent operations that do not conflict. In a serial schedule, one transaction finishes before the next begins.

Two operations **conflict** when they belong to different transactions, access the same data item, and at least one of them is a write. For example, if `T1` writes account A and `T2` then reads A, swapping those operations could change the balance `T2` reads. Two reads of A can be swapped, as can operations on different accounts, provided they belong to different transactions.

Using a dependency graph to test conflict serializability

A **dependency graph**, also called a **precedence graph**, shows which transactions must come before others to preserve the order of conflicting operations.

1. Draw one node for each transaction.
2. Scan the schedule from left to right. For every conflicting pair, draw an arrow from the transaction with the **earlier** operation to the transaction with the **later** operation. Two operations conflict if they belong to different transactions, access the same item, and at least one is a write.
3. Check for a **cycle**. If the graph has no cycle, the schedule is **conflict serializable**. A topological ordering of the graph gives an equivalent serial order. If the graph has a cycle, no serial order can satisfy all the arrows.

Example 1 — No cycle

$W_1(A),\ R_2(A),\ W_1(B),\ R_2(B)$

On both A and B, T1 writes before T2 reads. Both conflicts give the same arrow:

T1 → T2

The operations interleave, but the schedule is conflict equivalent to running **T1 completely before T2**.

Example 2 — A cycle

$W_1(A),\ R_2(A),\ W_2(B),\ R_1(B)$

- On **A**, $W_1(A)$ comes before $R_2(A)$, giving **T1 → T2**.
- On **B**, $W_2(B)$ comes before $R_1(B)$, giving **T2 → T1**.

The graph contains the cycle **T1 → T2 → T1**. A requires T1 to come first, while B requires T2 to come first, so the schedule fails the conflict-serializability test.

Example 3 — No cycle, with two possible serial orders

$R_1(A),\ R_2(A),\ W_3(A),\ R_1(B)$

- $R_1(A)$ comes before $W_3(A)$, giving **T1 → T3**.
- $R_2(A)$ comes before $W_3(A)$, giving **T2 → T3**.
- T1 and T2 both read A, so their reads create no edge.

The graph has no cycle. The schedule is **conflict serializable** with either **T1 → T2 → T3** or **T2 → T1 → T3** as its serial order.

Example 4 — A cycle on one data item

$R_1(A),\ W_2(A),\ W_1(A)$

- $R_1(A)$ comes before $W_2(A)$, giving **T1 → T2**.
- $W_2(A)$ comes before $W_1(A)$, giving **T2 → T1**.

The graph contains the cycle **T1 → T2 → T1**, so the schedule fails the conflict-serializability test.

Example 5 — A cycle involving three transactions

$W_1(A),\ R_2(A),\ W_2(B),\ R_3(B),\ W_3(C),\ R_1(C)$

- On **A**, $W_1(A)$ comes before $R_2(A)$, giving **T1 → T2**.
- On **B**, $W_2(B)$ comes before $R_3(B)$, giving **T2 → T3**.
- On **C**, $W_3(C)$ comes before $R_1(C)$, giving **T3 → T1**.

The graph contains the cycle **T1 → T2 → T3 → T1**. No serial order can satisfy all three edges.